ChatGPT

Implementation Conformance Analysis

1. Acceptance Algebra (Mathematical Fidelity)

SpecChain Part 1 defines acceptance states $q \in Q$ as a **typed product** of sum-axes S and max-axes M, with each axis carrying an identity element and an absorbing bottom $\perp$ [1]. The global aggregation $\wedge$ combines vectors coordinatewise: it *sums* on S-axes and *takes max* on M-axes, propagating $\perp$ (unknown) so that $x \otimes \perp = \perp$ [2] [3]. The Sirus code faithfully encodes this structure: its `Q` class carries axis sets `S` and `M`, uses a special string "⊥" for bottom, and implements two operations: `hetero_join` (horizontal aggregation) and `sequential` (vertical aggregation). In `hetero_join`, the code sums values on S-axes and takes max on M-axes, returning $\perp$ on any axis whenever either operand is $\perp$. In `sequential`, it adds along all axes (summing on both families) and likewise absorbs $\perp$. Thus unknowns are **fail-closed** under both aggregators as required [1] [2]. For example, in `hetero_join` the code does "if either is ⊥, result = ⊥; else out[axis] = a + b (for S) or max(a,b) (for M)". This exactly matches the spec's requirement that $\perp$ is an absorbing bottom and that sum-axes use addition while max-axes use maximum [1] [2]. In short, **Sirus implements the acceptance algebra Q correctly**: it enforces the coordinatewise product structure and aggregation laws (sum on S, max on M) and treats $\perp$ as absorbing.

Example: The spec gives $(q \wedge q')_i = q_i + q'_i$ for $i \in S$ and $\max\{q_i, q'_i\}$ for $i \in M$ [2]; the code's `hetero_join` loop exactly sums or maximizes accordingly.

2. Guard-First Discipline and Safety Checks

SpecChain Parts 3–5 insist on a **guard-first, fail-closed** execution: any row (frontier update) that has an unknown on a priced coordinate must be rejected **before** pricing or aggregation [4] [5]. In the Sirus loader code, each guard function (`guard_EF`, `guard_DRO`, etc.) first calls a helper that recursively checks if *any* input field equals a bottom sentinel. If so, it immediately returns a **FAIL** receipt with code `C_UNK_PRICED`. This matches SCp5's "Guard-first rule (fail-closed)": "No priced contribution may pass a guard if any of its priced inputs are ⊥... emit a RejectionCert" [5]. Indeed, Sirus rejects ⊥-containing rows at the guard stage, so ⊥ never propagates into the running q state, as required by Def.1.5 [4].

By contrast, **axis isolation** (SCp5 N4) demands that each adapter only write to its declared axes, with violations causing `C_AXIS_LEAK` errors [6]. We did not find code enforcing this policy: the loader has no explicit whitelist checks after adapters run. In other words, Sirus currently lacks an **axis-isolation check**. This is a notable omission, since SCp5 N4 expects any undeclared axis write to trigger a fatal rejection [6].

3. Runtime Checks (I1–I13) and No-Zeno Discipline

Part 3 specifies a suite of baseline checks (I1–I10) each emitting a uniform receipt, and tail checks (I11–I13). In Sirus, the only mechanized predicates we see are the guard checks (which align with I1–I5 in spirit). Other formal checks from SCp3 (like raw-data hash matching or evidence-bundle completeness) are **not**

implemented. Thus the code provides only the minimal guard receipts, but not the full I1–I10 gamut. All guard receipts use a uniform JSON schema (fields `predicate_id`, `inputs_digest`, `checker_hash`, `outcome.verdict`, and optionally a reason code) consistent with SCp5 N6 [7], so the **receipt format** is correct.

The loader also implements the **No-Zeno dwell counter** as described in Part 3. The `LoaderState` tracks a `dwell` counter (initialized by `Tmax` in config) and the function `no_zeno_step` resets or decrements it. If too many steps occur without a "strict decrease," the code raises a HALT. This enforces the requirement that the rank (and a finite dwell budget) prevents infinite loops [8]. Sirus emits a receipt on every dwell action with fields `"predicate_id": "NO_ZENO_WINDOW"`, `"window_size"`, `"post_dwell"`, and `"strict_decrease"`. This is close to the spec's **DwellReceipt** schema [9], though it omits the `pre_rank`/`post_rank` fields that SCp3 example shows. In short, Sirus **implements a windowed-progress rule** via the dwell counter and emits a matching receipt, but it does not include the full lexicographic rank in the record.

The lexicographic **rank itself** (state.rank) is present in the code but only loosely used: on any rejection, a flag `strict_decrease` resets the dwell, but the components of the rank vector are never explicitly updated as SCp3 demands for each rule. Thus, while Sirus enforces a *No-Zeno cap*, it does not fully manage a multi-component rank to guarantee strictly decreasing lexicographic order as per Def.2.3 [10]. Nevertheless, the core No-Zeno logic (reset on progress, halt when exhausted) is operational.

4. Loader Pipeline and Receipts

The spec requires a **fixed pipeline** EF→DRO→PR→CBF→Ω [11] [12]. Sirus enforces this: its `run_pipeline` function expects exactly 5 guard functions in that order and processes inputs stage by stage, halting on the first failure. At each stage a uniform guard receipt is emitted (in line with SCp5 N6), and on failure a RejectionCert is appended. Thus the loader is a simple verifiable automaton sequenced EF–Ω, matching SCp3 and SCp5's mandated order [11] [12].

All receipts use canon/v1 JSON and Blake3 digests as required (the code's `b3_json` ensures this). Each guard uses the kernel's code hash (checker_hash) and includes an inputs-digest of the canonical payload, as SCp5 N6 specifies [7]. For example, a PASS receipt from `guard_EF` will have the fields `predicate_id: "EF_LOCALITY"`, the BLAKE3 hash of the inputs, `checker_hash`, and `"outcome": {"verdict": "PASS"}`. On a FAIL (e.g. unknown), it adds `"reason_code":"C_UNK_PRICED"`. This uniform receipt format is fully compliant with the spec [7].

Notably missing are certain Part 5 constructs: we found no implementation of *RetuneCert* or *UpdateCert* issuance, nor a *TailReceipt* format beyond the No-Zeno receipt. Sirus does include a rudimentary tail-contract interface and one family (I11 Contraction), but it does not appear to canonicalize tail witnesses via a TailReceipt or bind them by Blake3 to the proof surface as SCp4 requires. Moreover, there is no code enforcing *EnvLock/ReplayPlan* determinism beyond using a fixed RNG seed in config. In summary, the loader semantics (stage sequencing, guard receipts, and dwell check) are correctly implemented; but advanced features like UpdateCert binding, proof-surface binding for new receipts, and strict replay determinism are not evident.

5. Tail Contracts and Acceptance Extensionality

SpecChain Part 4 defines **TailContracts** (I11–I13) to handle late-stage predicates based only on a tail witness. The code includes a base `TailContract` interface and one concrete family `I11_Contraction`. The `run()` method in `I11` checks its hypothesis (e.g. FP-contraction bound) and returns a verdict with `"verdict":"ACCEPT"` or `"REJECT"`. Crucially, it depends only on the tail's witness data and the pinned policy (no external state), so it *is* extensional in the tail bytes, as required by Theorem 1.4 [13]. However, the code does not explicitly produce a *TailReceipt* bound into the proof surface — it simply returns a Python dict. Thus acceptance of a tail contract is not currently recorded as a receipt with Blake3 binding, contrary to SCp4's mandate that every successful tail run "emits receipts and ledger updates" [14].

In terms of correctness: if Sirus were extended to invoke these tail contracts at the final Ω stage, the existing implementation appears *logically aligned*: it would accept exactly when the tail predicate is met, and reject otherwise. This satisfies the safety-or-halt guarantee (Thm.1.6 [15]) in principle. But the machinery to *trigger* I11–I13, track their work and rho measures, and update the bundle manifest is not present. As of now, tail contracts in the code are placeholders and do not fully enforce Σ-non-expansion or produce the canonical TailReceipt expected by the spec.

Summary: The Sirus code faithfully realizes the core algebraic model (Q with ⊥ bottoms, coordinatewise aggregation) [1] [3] and correctly enforces guard-first rejection of unknowns [4] [5]. It emits canonical receipts for each guard and enforces an EF→Ω pipeline [11] [7]. No-Zeno liveness is implemented via a dwell counter, though the full lexicographic rank is not tracked in receipts [9]. Missing elements include axis-isolation checks [6], the full suite of baseline checks (I1–I10), and the tail-certification machinery (TailReceipt, UpdateCert) prescribed by Parts 4–5. In sum, Sirus covers the **normative core** (typed Q-algebra and guard receipts) but omits several compliance and governance features outlined in the spec.

Sources: SpecChain Parts 1–5 [1] [2] [11] [4] [5] [6] [7] [13] [9]. (All quotations are from the formal SpecChain papers; code behavior was checked against these requirements.)

---

[1] [2] SCp1.pdf
file://file-4siML99Sd3SYiedJnK1J8Z

[3] [5] [6] [7] [12] SCp5.pdf
file://file-2xbXGTnKyVihyR4Fmrggy9

[4] [8] [9] [10] [11] SCp3.pdf
file://file-H4YHhQUmfNw7s3ULfQpFPu

[13] [14] [15] SCp4.pdf
file://file-29deSYvbi6ywhEDkTL5wfC